

Problem *Polyline Simplification*

Projekat iz predmeta Geometrijski algoritmi
Matematički fakultet, Univerzitet u Beogradu

Boško Andrić, 1012/2025
mi251012@alas.matf.bg.ac.rs

17. septembar 2026.

Sadržaj

1	Uvod i opis problema	1
2	Geometrijski princip	1
2.1	Dvostruka površina trougla	1
2.2	Lokalni efekat izbacivanja	2
3	Struktura programa i algoritam	2
3.1	Strukture podataka za $O(n \log n)$ složenost	2
3.2	Koraci algoritma	2
4	Složenost	2
5	Prikaz u aplikaciji	2
6	Zaključak	2

1 Uvod i opis problema

Smanjivanje broja tačaka na krivim linijama je čest zadatak u kartografiji i obradi geografskih podataka. Problem *Polyline Simplification* sa Kattis platforme [1] traži da se iz izlomljene linije date nizom tačaka $P = (p_0, p_1, \dots, p_n)$ izbace manje važne tačke. Cilj je da od početnih $n+1$ temena ostane tačno m segmenata, a da linija zadrži svoj prepoznatljiv oblik.

U ovom projektu primenjen je Visvalingam–Whyatt algoritam [2]. Važnost svake unutrašnje tačke računa se na osnovu površine trougla koji ona pravi sa svojim susedima. U svakom koraku izbacuje se tačka sa najmanjom površinom trougla. Ako više tačaka ima istu površinu, prva se izbacuje ona sa manjim indeksnim brojem.

Ako bismo u svakom koraku tražili najmanju površinu prolaskom kroz sve tačke, složenost bi bila $O(n^2)$. Korišćenjem pametnijih struktura podataka, taj posao smanjujemo na vremensku složenost $O(n \log n)$ uz memorijsku složenost $O(n)$.

2 Geometrijski princip

2.1 Dvostruka površina trougla

Za unutrašnju tačku $p_i = (x_i, y_i)$ posmatramo trougao koji ona čini sa susedima p_{i-1} i p_{i+1} .

Definicija 2.1 (Površina trougla). Važnost tačke p_i jednaka je površini trougla $\triangle(p_{i-1}, p_i, p_{i+1})$. Da bismo izbegli rad sa razlomcima i deljenjem, računamo dvostruku površinu pomoću vektorskog proizvoda:

$$\mathcal{A}(p_{i-1}, p_i, p_{i+1}) = |(x_i - x_{i-1})(y_{i+1} - y_{i-1}) - (y_i - y_{i-1})(x_{i+1} - x_{i-1})|$$

Sva poređenja radimo direktno preko ovih celobrojnih vrednosti $\mathcal{A}$, što ubrzava algoritam i izbegava greške pri zaokruživanju.

2.2 Lokalni efekat izbacivanja

Efikasnost algoritma oslanja se na činjenicu da brisanje jedne tačke utiče samo na njene prve susede:

Teorema 2.1 (Lokalno osvežavanje). Uklanjanje tačke p_i menja trouglove samo za njenog prethodnika $p_{\text{prev}(i)}$ i njenog sledbenika $p_{\text{next}(i)}$. Površine ostalih trouglova ostaju potpuno iste.

Dokaz. Kada obrišemo tačku p_i , spajaju se njeni susedi p_L i p_R . Zbog toga se menja trougao za p_L i trougao za p_R . Pošto se ostale tačke i njihove veze nisu menjale, njihove površine ostaju nepromenjene. $\square$

Zahvaljujući ovome, posle svakog brisanja ažuriramo površine za samo dve susedne tačke u $O(\log n)$ vremenu, umesto da računamo sve ponovo.

3 Struktura programa i algoritam

Program je napisan u C++17 jeziku koristeći Qt 6 za prikaz. Logika algoritma nalazi se u `src/efficientsimplifier.cpp`, dok se prikaz i prozor nalaze u `src/canvas.cpp` i `src/mainwindow.cpp`.

3.1 Strukture podataka za $O(n \log n)$ složenost

Za postizanje $O(n \log n)$ složenosti upotrebljene su dve strukture:

1. **Dvostruko povezana lista (preko nizova):** Nizovi `prev_neighbor` i `next_neighbor` služe da u $O(1)$ vremenu nađemo i prevezemo susede kada se tačka obriše.
2. **Prioritetni skup (`std::set`):** Čuva indekse tačaka i njihove površine. Skup je sortiran primarno po površini (od najmanje ka najvećoj), a sekundarno po indeksu tačke.

3.2 Koraci algoritma

Algoritam 1 prikazuje jednostavan tok simplifikacije.

4 Složenost

Korak	Složenost	Opis
Inicijalizacija i prve površine	$O(n)$	Prolazak kroz sve tačke jednim nizom
Pravljenje <code>std::set</code> stablom	$O(n \log n)$	Ubacivanje $n - 1$ elemenata u skup
Uzimanje najmanje površine	$O(\log n)$	Brisanje prvog elementa iz skupa
Ažuriranje 2 susedne tačke	$O(\log n)$	Izbacivanje i ponovno ubacivanje u skup
Ukupno za sve izbačene tačke	$O(n \log n)$	Mnogo brže od naivnog $O(n^2)$ pristupa
Utoršak memorije	$O(n)$	Čuvanje nizova suseda i stanja u skupu

Naivni algoritam za $n = 100.000$ tačaka pravi oko 10^{10} računica, dok ova optimizovana verzija završava posao za manje od sekunde.

5 Prikaz u aplikaciji

Strogi poredak u `std::set` strukturi garantuje da će se uvek izbaciti tačna tačka propisana algoritmom.

Aplikacija podržava dva načina prikaza:

- **Algoritamski mod:** Prikazuje originalnu liniju sivom isprekidanom linijom, trenutnu liniju plavom bojom, a trougao koji se sledeći izbacuje označava crvenom bojom.
- **Planinski mod (LOD):** Prikazuje liniju kao reljef planine. Kako se tačke izbacuju, slika se postepeno smanjuje i skalira, dajući efekat odaljavanja kamere.

6 Zaključak

U projektu je uspešno napravljen Visvalingam–Whyatt algoritam za simplifikaciju polilinja u vremenu $O(n \log n)$. Spajanjem dvostruko povezane liste i balansiranog stabla postignuta je velika brzina rada. Grafički interfejs u Qt 6 okruženju omogućava jednostavan prikaz koraka algoritma i vizuelni efekat smanjenja detalja.

Algoritam 1 Simplifikacija poliliniije u $O(n \log n)$ vremenu

```
1: function VISVALINGAMSIMPLIFY( $P = (p_0, \dots, p_n), m$ )
2:   Inicijalizuj nizove prev_neighbor i next_neighbor
3:    $Q \leftarrow \emptyset$  ▷ Prioritetni skup std::set
4:   for  $i \leftarrow 1$  to  $n - 1$  do
5:      $\text{area}[i] \leftarrow \text{ComputeDoubleArea}(p_{i-1}, p_i, p_{i+1})$ 
6:      $Q.\text{insert}(\{i, \text{area}[i]\})$ 
7:   end for
8:    $\text{count} \leftarrow n + 1$ 
9:   while  $\text{count} > m + 1$  and  $Q \neq \emptyset$  do
10:     $\text{min\_elem} \leftarrow Q.\text{extract\_min}()$  ▷ Uzimamo najmanju površinu u  $O(\log n)$ 
11:     $\text{idx} \leftarrow \text{min\_elem.index}$ ,  $L \leftarrow \text{prev}[\text{idx}]$ ,  $R \leftarrow \text{next}[\text{idx}]$ 
12:     $\text{next}[L] \leftarrow R$ ,  $\text{prev}[R] \leftarrow L$  ▷ Prevezivanje suseda u  $O(1)$ 
13:    Sačuvaj korak u istoriju za animaciju
14:    if  $L > 0$  then ▷ Ažuriraj levog suseda
15:       $Q.\text{erase}(\{L, \text{area}[L]\})$ 
16:       $\text{area}[L] \leftarrow \text{ComputeDoubleArea}(p_{\text{prev}[L]}, p_L, p_R)$ 
17:       $Q.\text{insert}(\{L, \text{area}[L]\})$ 
18:    end if
19:    if  $R < n$  then ▷ Ažuriraj desnog suseda
20:       $Q.\text{erase}(\{R, \text{area}[R]\})$ 
21:       $\text{area}[R] \leftarrow \text{ComputeDoubleArea}(p_L, p_R, p_{\text{next}[R]})$ 
22:       $Q.\text{insert}(\{R, \text{area}[R]\})$ 
23:    end if
24:     $\text{count} \leftarrow \text{count} - 1$ 
25:  end while
26:  return istorija koraka
27: end function
```

Literatura

- [1] Kattis, *Polyline Simplification*, <https://open.kattis.com/problems/polyline-simplification>.
- [2] M. Visvalingam, J. D. Whyatt, *Line simplification by repeated elimination of points*, The Cartographic Journal, 30(1), 46-51, 1993.
- [3] P. Janičić, *Računarska geometrija*, Matematički fakultet u Beogradu, 2025.
- [4] T. H. Cormen et al., *Introduction to Algorithms*, MIT Press, 2009.
- [5] Qt Group, *Qt 6 Documentation*, <https://doc.qt.io/qt-6/>.